

Documentacion Tecnica

EcoEmpack — Aplicacion Web Full Stack de Empaques Ecologicos

Proyecto Fase 1 — Desarrollo Web con Node.js, Express y MongoDB

Proyecto	Version	Fase	Año
EcoEmpack	1.0.0	1 de N	2024

Stack

Node.js + Express + MongoDB + JWT

📋 Tabla de Contenidos

1. Descripcion General del Proyecto
2. Arquitectura de la Aplicacion
3. Tecnologias y Dependencias
4. Estructura de Archivos
5. Frontend — Descripcion Detallada
6. Backend — Descripcion Detallada
7. Modelos de Base de Datos
8. Endpoints de la API RESTful
9. Sistema de Autenticacion JWT

10. Instrucciones de Instalacion y Ejecucion
11. Glosario y Explicaciones "For Dummies"

1. Descripcion General del Proyecto

EcoEmpack es una aplicacion web full stack desarrollada para una empresa de empaques ecologicos cuya mision es ayudar a las personas a realizar un *detox del plastico* y protegerse de los microplasticos. La aplicacion presenta el catalogo de productos, informacion cientifica sobre microplasticos y un sistema de administracion con autenticacion segura.

Caracteristica	Detalle
Tipo de aplicacion	Web Full Stack (SPA parcial + API REST)
Entorno del backend	Node.js v18+ con Express.js v4
Base de datos	MongoDB (con Mongoose ODM)
Autenticacion	JWT (JSON Web Tokens) + bcryptjs
Frontend	HTML5 + CSS3 + JavaScript Vanilla (ES6+)
Puerto del servidor	3000 (configurable via .env)

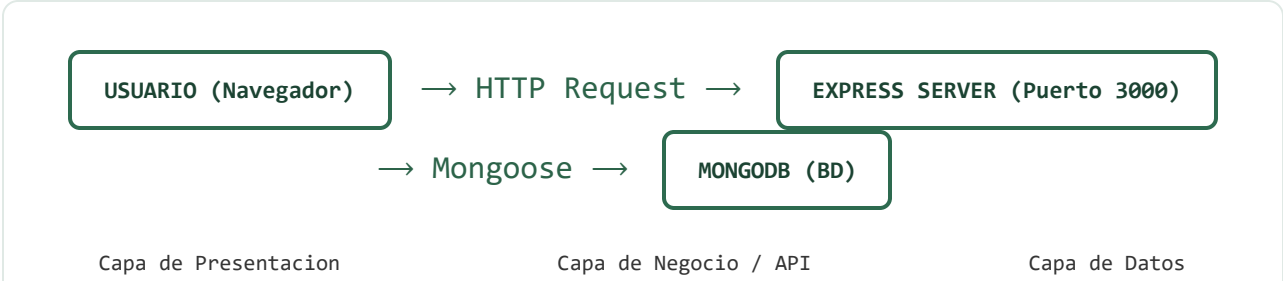
Objetivos del Proyecto

- Configurar un entorno de desarrollo Node.js con npm y variables de entorno con dotenv
- Desarrollar un frontend responsivo con HTML, CSS y JavaScript
- Construir una API RESTful en Express.js con operaciones CRUD completas
- Implementar autenticacion basada en JWT con contrasenas encriptadas con bcrypt
- Conectar el backend a una base de datos MongoDB mediante Mongoose
- Desplegar una version preliminar en entorno local y en la nube

---

2. Arquitectura de la Aplicacion

La aplicacion sigue una arquitectura **cliente-servidor de tres capas**:



Flujo de una peticion autenticada

```
/* Ejemplo: Admin crea un nuevo producto */

1. Frontend envia:
   POST /api/products
   Headers: { Authorization: "Bearer eyJhbGciOiJIUzI1NiJ9..." }
   Body: { nombre: "Bolsas Kraft", precio: 0.45 }

2. Middleware auth.js intercepta la peticion:
   - Extrae el token del Header
   - jwt.verify(token, JWT_SECRET) → decodifica el payload
   - Busca el usuario en MongoDB con decoded.id
   - Agrega req.usuario = { id, email, rol: "admin" }
   - Llama next() para continuar

3. Middleware soloAdmin verifica rol:
   - req.usuario.rol === "admin" → next()

4. Route handler crea el producto:
   - Product.create(req.body)
   - MongoDB guarda el documento
   - Responde con { success: true, product: {...} }
```

3. Tecnologias y Dependencias

v18.x LTS

Node.js

Entorno de ejecucion de JavaScript en el servidor. Permite correr JS fuera del navegador.

v4.18.2

Express.js

Framework minimalista para Node.js. Facilita la creacion de servidores y rutas HTTP.

v8.0.3

Mongoose

ODM para MongoDB. Permite definir schemas, validaciones y metodos en los modelos.

v9.0.2

jsonwebtoken

Genera y verifica tokens JWT para la autenticacion stateless de usuarios.

v2.4.3

bcryptjs

Encripta contrasenias con hashing unidireccional. Imposible revertir el hash.

v16.3.1

dotenv

Carga variables de entorno desde el archivo .env para proteger credenciales.

v2.8.5

cors

Habilita Cross-Origin Resource Sharing para que el frontend pueda comunicarse con el backend.

HTML5 + CSS3

Estructura y estilos del frontend. CSS Variables, Flexbox, Grid y animaciones.

JavaScript ES6+

Logica del frontend: fetch API, async/await, DOM manipulation, localStorage.

package.json — Dependencias

```
{
  "dependencies": {
    "express":       "^4.18.2",   // Servidor HTTP
    "mongoose":      "^8.0.3",    // Conexion a MongoDB
    "jsonwebtoken":  "^9.0.2",    // Autenticacion JWT
    "bcryptjs":      "^2.4.3",    // Encriptacion de contrasenias
    "dotenv":        "^16.3.1",   // Variables de entorno
    "cors":          "^2.8.5"     // Cross-Origin Resource Sharing
  },
  "devDependencies": {
    "nodemon":       "^3.0.2"     // Reinicio automatico en desarrollo
  }
}
```

---

4. Estructura de Archivos del Proyecto

EcoEmpack/ │ ├── **frontend/** ← Capa de presentacion (cliente) │ ├── index.html ← Pagina principal: Hero, Productos, Detox, Nosotros │ ├── login.html ← Pagina de autenticacion: Login y Registro │ ├── **css/** │ │ ├── style.css ← Estilos globales: layout, componentes, responsive │ │ └── login.css ← Estilos especificos del formulario de login │ └── **js/** │ ├── catalog.js ← Datos de 6 productos + logica del modal con tabs │ ├── main.js ← Navbar, animaciones, helper apiRequest() │ └── login.js ← Formularios login/registro + gestion de JWT │ ├── **backend/** ← Capa de negocio y datos (servidor) │

```
├── server.js ← Punto de entrada: Express + MongoDB + Rutas
│ ├── package.json ← Dependencias y scripts de NPM
│ ├── .env.example ← Plantilla de variables de entorno (SIN valores reales)
│ ├── .gitignore ← Excluye node_modules/ y .env de Git
│ ├── models/ ← Schemas de Mongoose para MongoDB
│ │ ├── User.js ← Schema de usuario + hash bcrypt en pre-save hook
│ │ └── Product.js ← Schema de producto ecologico
│ ├── routes/ ← Handlers de endpoints HTTP
│ │ ├── auth.js ← POST /register, POST /login, GET /perfil
│ │ └── products.js ← CRUD completo de productos
│ └── middleware/ ← Funciones intermedias reutilizables
│ ├── auth.js ← protegerRuta (JWT) + soloAdmin (rol)
│ └── errorHandler.js ← Manejador global de errores con tipos
└── GUIA_DUMMIES.md ← Documentacion explicativa para principiantes
```

---

5. Frontend — Descripcion Detallada

index.html — Pagina Principal

Contiene cuatro secciones organizadas con navegacion fija (`sticky navbar`):

Seccion	ID	Descripcion
Navegacion	`.navbar`	Logo, links de navegacion, boton de login. Cambia estilos al hacer scroll.
Hero	`#inicio`	Titulo principal, estadisticas, tarjetas flotantes animadas con CSS.
Productos	`#productos`	Grid de 6 tarjetas generadas dinamicamente por `catalog.js` . Cada una abre un modal.
Detox	`#detox`	3 tarjetas informativas + grafico de barras de fuentes de exposicion a microplasticos.
Nosotros	`#nosotros`	Historia y valores de la empresa con indicadores clave.

catalog.js — Sistema de Catalogo Modal

Contiene el array `products[]` con los datos de los 6 productos y la funcion `openModal(id)` que:

1. Encuentra el producto por ID en el array
2. Construye dinamicamente el HTML de las 3 pestanas (Catalogo / Uso / Eco)
3. Inserta el contenido en el modal del DOM
4. Activa la clase `.active` en el overlay para mostrarlo

login.js — Autenticacion en el Frontend

Maneja dos formularios mediante `fetch()` + `async/await`:

- **Login:** Envia `POST /api/auth/login` → guarda el JWT en `localStorage`
- **Registro:** Envia `POST /api/auth/register` → redirige al login
- **Modo demo:** Si el backend no esta corriendo, acepta `admin@ecoempack.com / admin123` localmente para pruebas

---

6. Backend — Descripcion Detallada

server.js — Configuracion Central

```
// Orden de configuracion en server.js:
1. dotenv.config()                    // Carga variables de .env
2. app.use(cors({...}))               // Permite peticiones del frontend
3. app.use(express.json())            // Parsea body en formato JSON
4. app.use('/api/auth', authRoutes)   // Registra rutas de auth
5. app.use('/api/products', prodRoutes)// Registra rutas de productos
6. app.use(errorHandler)             // Manejador global (SIEMPRE al final)
7. mongoose.connect(MONGO_URI)       // Conecta a la BD y arranca el servidor
```

middleware/auth.js — Verificacion JWT

El middleware `protegerRuta` se ejecuta antes de cada ruta protegida:

```
// 1. Leer el token del header Authorization: Bearer <token>
// 2. jwt.verify(token, JWT_SECRET) → lanza error si expirado/invalido
// 3. User.findById(decoded.id) → confirmar que el usuario existe en BD
// 4. req.usuario = usuario → disponible en todas las rutas siguientes
// 5. next() → continuar hacia el route handler
```

middleware/errorHandler.js — Errores Tipados

Tipo de error	Codigo HTTP	Descripcion
ValidationError (Mongoose)	400	Datos invalidos segun el schema
Codigo 11000 (duplicate key)	400	Email ya registrado

JsonWebTokenError	401	Token malformado
TokenExpiredError	401	Token expirado
Error generico	500	Error interno del servidor

7. Modelos de Base de Datos

Modelo User (models/User.js)

Campo	Tipo	Requerido	Descripcion
`nombre`	String	Si	Nombre del usuario, con trim()
`apellido`	String	Si	Apellido del usuario
`email`	String	Si	Unico, lowercase, validado con regex
`password`	String	Si	Minimo 8 chars, hasheado con bcrypt (select: false)
`rol`	Enum	No	'usuario' (default) | 'admin'
`activo`	Boolean	No	true por defecto. Soft-delete si se pone false
`createdAt`	Date	Auto	Generado por timestamps: true
`updatedAt`	Date	Auto	Generado por timestamps: true

> **Hook pre-save:** Antes de guardar, si la contrasena fue modificada, se encripta con `bcrypt.hash(password, 10)` . El numero 10 indica los "rounds" de encriptacion (mayor = mas seguro pero mas lento). Esto ocurre automaticamente, sin codigo extra en las rutas.

Modelo Product (models/Product.js)

Campo	Tipo	Descripcion
`nombre`	String	Nombre del producto (requerido)
`descripcion`	String	Descripcion detallada

`categoria`	Enum	bolsas | cajas | film | contenedores | vidrio | otro
`precio`	Number	Precio unitario, minimo 0
`stock`	Number	Unidades disponibles
`materiales`	[String]	Array de materiales de fabricacion
`certificaciones`	[String]	Array de certificaciones ecologicas
`biodegradable`	Boolean	true por defecto
`tiempoBiodegradacion`	String	Ej: "2-6 semanas"
`activo`	Boolean	Soft-delete: false = oculto

8. Endpoints de la API RESTful

URL base: `http://localhost:3000/api` | Produccion: `https://ecoempack-api.onrender.com/api`

Autenticacion — /api/auth

POST **/api/auth/register** Publica — Crear nueva cuenta

Body (JSON): `{ "nombre": "Ana", "apellido": "Lopez", "email": "ana@test.com", "password": "miPass123" }`

Respuesta 201: `{ "success": true, "token": "eyJhbGci...", "user": { "id", "nombre", "email", "rol" } }`

Error 400: Si el email ya existe o los datos son invalidos.

POST **/api/auth/login** Publica — Iniciar sesion

Body (JSON): `{ "email": "ana@test.com", "password": "miPass123" }`

Respuesta 200: `{ "success": true, "token": "eyJhbGci...", "user": {...} }`

Error 401: Credenciales incorrectas.

GET `/api/auth/perfil` **Protegida — JWT** Ver mi perfil

Header requerido: `Authorization: Bearer <token>`

Respuesta 200: `{ "success": true, "user": { "nombre", "email", "rol" } }`

Productos — /api/products

GET `/api/products` **Publica** Listar todos los productos

Query params opcionales: `?categoria=bolsas` `?biodegradable=true`

Respuesta 200: `{ "success": true, "count": 6, "products": [...] }`

GET `/api/products/:id` **Publica** Ver un producto por ID

Params: `:id` = ObjectId de MongoDB (ej: `507f1f77bcf86cd799439011`)

Respuesta 200: `{ "success": true, "product": { ... } }`

Error 404: Producto no encontrado.

POST `/api/products` **Protegida — Admin** Crear producto

Header: `Authorization: Bearer <token-admin>`

Body: `{ "nombre", "descripcion", "categoria", "precio", "stock", "biodegradable" }`

Respuesta 201: `{ "success": true, "product": { _id, nombre, ... } }`

PUT `/api/products/:id` **Protegida — Admin** Actualizar producto

Header: `Authorization: Bearer <token-admin>`

Body: Solo los campos a actualizar: `{ "precio": 200 }`

Respuesta 200: `{ "success": true, "product": { ...actualizado } }`

DELETE `/api/products/:id` **Protegida — Admin** Eliminar producto (soft)

Header: `Authorization: Bearer <token-admin>`

Nota: Implementa *soft delete*: cambia `activo: false` en vez de borrar el registro. Esto preserva el historial de datos.

Respuesta 200: `{ "success": true, "message": "Producto eliminado correctamente." }`

---

9. Sistema de Autenticacion JWT

El sistema usa autenticacion **stateless** basada en JWT. No se almacenan sesiones en el servidor.

Flujo completo del ciclo de autenticacion

```
/* REGISTRO */
POST /api/auth/register
  → Validar datos de entrada
  → User.create() → pre-save hook encripta la contrasena con bcrypt
  → jwt.sign({ id: usuario._id }, JWT_SECRET, { expiresIn: '7d' })
  → Responder con token + datos del usuario

/* LOGIN */
POST /api/auth/login
  → User.findOne({ email }).select('+password')
  → bcrypt.compare(passwordIngresada, passwordHash) → boolean
  → Si correcto: jwt.sign() → devolver token
  → Si incorrecto: 401 Unauthorized

/* PETICION PROTEGIDA */
GET|POST|PUT|DELETE /api/products
  → Header: Authorization: Bearer <token>
  → middleware/auth.js:
      jwt.verify(token, JWT_SECRET) → { id, iat, exp }
      User.findById(decoded.id) → confirmar usuario activo
      req.usuario = usuario
      next()
  → Route handler ejecuta la logica
```

> **Seguridad de las contrasenias:** bcrypt aplica un algoritmo de hashing con "salt" aleatorio. La misma contrasena produce hashes diferentes en cada ejecucion, lo que impide ataques de tabla arcoiris (rainbow tables). El parametro `rounds=10` hace que un ataque de fuerza bruta sea computacionalmente impracticable.

> **Variables de entorno criticas:** `JWT_SECRET` y `MONGODB_URI` jamas deben estar en el codigo fuente. Se almacenan exclusivamente en el archivo `.env` que esta incluido en `.gitignore` .

---

10. Instrucciones de Instalacion y Ejecucion

Requisitos Previos

- Node.js v18 o superior (nodejs.org)
- MongoDB local o cuenta en MongoDB Atlas (mongodb.com/atlas)
- Git instalado y cuenta en GitHub

Instalacion Local

```
# 1. Navegar a la carpeta del backend
cd C:\Users\TuUsuario\Desktop\EcoEmpack\backend

# 2. Instalar dependencias
npm install

# 3. Crear archivo de entorno (copiar la plantilla)
copy .env.example .env
# Luego editar .env con tus datos (MongoDB URI y JWT Secret)

# 4. Iniciar el servidor en modo desarrollo
npm run dev

# Resultado esperado:
✅ Conectado a MongoDB correctamente
🚀 Servidor corriendo en http://localhost:3000
```

Variables de Entorno (.env)

```
PORT=3000
MONGODB_URI=mongodb://localhost:27017/ecoempack
JWT_SECRET=clave_secreta_larga_y_unica_aqui
JWT_EXPIRES_IN=7d
NODE_ENV=development
```

Prueba rapida de la API

Abrir el navegador en `http://localhost:3000` debe mostrar:

```
{
  "message": "Bienvenido a la API de EcoEmpack",
  "version": "1.0.0"
}
```

> **Herramienta recomendada para probar la API:** Descarga **Thunder Client** (extension gratuita de VS Code) o **Postman** para enviar peticiones GET, POST, PUT, DELETE sin necesidad del frontend.

📚 11. Guía Explicativa "For Dummies"

Esta sección explica cada tecnología y concepto del proyecto en lenguaje simple, con analogías del mundo real. No se requiere experiencia previa.

🆕 ¿Qué estamos construyendo?

Una **aplicación web full stack** tiene dos partes que trabajan juntas:

> 🍽️ **Anología del restaurante:**
> • El **Frontend** = el salón del restaurante (lo que ve el cliente)
> • El **Backend** = la cocina (donde se prepara todo, invisible al cliente)
> • La **Base de Datos** = la despensa (donde se guardan los ingredientes / datos)
> • La **API** = el mesero (quien lleva mensajes entre cocina y salón)

💻 Tecnologías explicadas una por una

Tecnología	Qué es en palabras simples	Analogía
Node.js	JavaScript corriendo en tu computadora, no en el navegador. Permite usar JS para crear el servidor.	Es como sacar a JS del navegador y ponerlo a trabajar como empleado en la cocina.
Express.js	Un framework que facilita crear servidores y rutas con muy poco código.	Node.js es una caja de herramientas. Express es el cinturón organizado donde cada herramienta ya tiene su lugar.

MongoDB	Base de datos que guarda información en formato JSON (como objetos de JavaScript), sin tablas fijas.	MySQL es una hoja de Excel muy rígida. MongoDB es una colección de tarjetas de notas donde cada tarjeta puede tener diferente información.
Mongoose	Puente entre Node.js y MongoDB. Permite definir "plantillas" (schemas) para que los datos siempre tengan el formato correcto.	Si MongoDB es una bodega sin orden, Mongoose es el sistema de organización que dice: "aquí van los zapatos, allá los libros".
JWT	Un "pasaporte digital" que el servidor le da al usuario después de iniciar sesión. En cada petición protegida se muestra ese pasaporte.	Cuando entras a un parque de diversiones, pagas y te ponen una pulsera. JWT es esa pulsera: la muestras en cada atracción sin volver a pagar.
bcryptjs	Convierte la contraseña "hola123" en algo como "$2a$10$Kxm..." que es imposible de revertir. Las contraseñas nunca se guardan en texto plano.	Es como pasar tu mensaje por una picadora de papel. Puedes confirmar que algo fue picado, pero no reconstruirlo.
dotenv	Lee un archivo `.env` con información secreta (contraseñas, claves) y la inyecta como variables. Ese archivo **nunca** se sube a GitHub.	En vez de publicar tus llaves en Facebook, las guardas en un llavero privado y dot env las recupera solo cuando tu programa las necesita.
CORS	Permiso que da el servidor para aceptar peticiones de otro origen (por ejemplo: el frontend en puerto 5500 hablando con el backend en puerto 3000).	Sin CORS, el navegador bloquea las peticiones entre dominios/puertos diferentes como medida de seguridad. CORS es la lista de invitados que da acceso.

🔁 ¿Qué es una API RESTful?

API = intermediario que permite que dos programas se comuniquen. **REST** = conjunto de reglas para diseñar esa comunicación usando verbos HTTP:

Verbo HTTP	Qué significa	Ejemplo en este proyecto
GET	Obtener / Leer información	Ver todos los productos del catálogo
POST	Crear / Enviar información nueva	Registrar un nuevo usuario o producto
PUT	Actualizar información existente	Cambiar el precio de un producto
DELETE	Eliminar información	Desactivar un producto del catálogo

📊 ¿Qué son las operaciones CRUD?

> **CRUD** = **C**reate, **R**ead, **U**pdate, **D**elete — las 4 operaciones básicas de cualquier sistema que maneja datos.
>
> Toda aplicación de gestión (tienda, red social, banco) en el fondo hace solo estas 4 cosas con sus datos. En este proyecto: crear productos, leerlos, editarlos y eliminarlos.

🔒 ¿Cómo funciona JWT paso a paso?

```
/* Paso 1 */ Usuario escribe email + contraseña en el formulario
/* Paso 2 */ Frontend envía POST /api/auth/login al backend
/* Paso 3 */ Backend busca el usuario en MongoDB
/* Paso 4 */ bcrypt.compare(pass_ingresada, hash_en_BD) → true o false
/* Paso 5 */ Si es correcto: jwt.sign() genera el token y lo envía
/* Paso 6 */ Frontend guarda el token en localStorage
/* Paso 7 */ En peticiones futuras el frontend envía: Authorization: Bearer <token>
/* Paso 8 */ Middleware auth.js verifica el token antes de dar acceso
```

📄 ¿Por qué usamos dotenv y variables de entorno?

> ⚠️ **Problema:** Si escribes tu contraseña de la base de datos directamente en el código y lo subes a GitHub, **cualquier persona en el mundo puede verla**.
>
> ✅ **Solución:** Guardas los datos sensibles en un archivo `.env` que nunca se sube a GitHub (está en `.gitignore`). El código solo lee `process.env.MONGODB_URI` y dotenv lo conecta automáticamente con el valor real del `.env`.

🗄️ ¿Qué es `localStorage`?

Es una pequeña base de datos que tiene cada navegador. Permite guardar texto (como el token JWT) que persiste aunque cierres la pestaña. Se accede con:

```
localStorage.setItem('token', jwt) // guardar
localStorage.getItem('token') // leer
localStorage.removeItem('token') // borrar (logout)
```

🏠 ¿Qué es un puerto?

> Tu computadora tiene miles de "puertas" numeradas (puertos). Cada servicio usa una diferente:
>
> • Puerto **80** → sitios web normales (HTTP)

> • Puerto **443** → sitios web seguros (HTTPS)
> • Puerto **3000** → nuestro servidor Express
> • Puerto **5500** → Live Server de VS Code (el frontend)
>
> Por eso necesitamos CORS: el frontend en `:5500` quiere hablarle al backend en `:3000`, y el navegador por defecto bloquea esa comunicación entre puertos diferentes.

📚 Glosario rápido de términos

Término	Significado simple
API	Intermediario que permite que dos programas se comuniquen
Backend	La parte del servidor que el usuario no ve
bcrypt	Herramienta que convierte contraseñas en códigos ilegibles e irreversibles
CORS	Permiso del servidor para recibir peticiones de otros orígenes
CRUD	Create, Read, Update, Delete — las 4 operaciones básicas de datos
dotenv	Herramienta para leer variables secretas de un archivo `.env`
endpoint	URL específica de la API que realiza una tarea concreta
Express	Framework para crear servidores con Node.js fácilmente
Frontend	La parte visual del sitio que ve el usuario
Full Stack	Aplicación que tiene frontend + backend en un mismo proyecto
JWT	Token de autenticación que funciona como “pasaporte digital”
localhost	Tu propia computadora (dirección IP 127.0.0.1)
Middleware	Función que corre entre la petición y la respuesta del servidor
MongoDB	Base de datos que guarda datos en formato JSON (sin tablas)
Mongoose	Puente entre Node.js y MongoDB con validaciones y schemas
Node.js	JavaScript corriendo fuera del navegador, en el servidor
npm	Gestor de paquetes de Node.js. Descarga librerías con `npm install`
package.json	Lista de dependencias y configuración del proyecto Node.js

Puerto	“Puerta” numerada por donde entra/sale información en red
REST	Estándar para diseñar APIs usando verbos HTTP (GET, POST, PUT, DELETE)
Route	“Camino” que toma una petición HTTP según su URL y método
Schema	Plantilla que define cómo deben ser los datos antes de guardarlos
Token	Cadena de texto cifrada que actúa como credencial de identidad